First: what are we predicting?

Identify an Iris flower's species from its petal measurements. A label is its known species.

The famous Iris dataset has 150 flowers and three species: setosa, versicolor, and virginica. Scikit-learn includes it as load_iris(). For a manageable hand example, we select five real flowers from just versicolor and virginica.

Inputs: two measurements in centimeters (cm)

Petal length measures base to tip. Petal width measures across the widest part. A length of 4.6 means 4.6 cm, not a score. The next page shows exactly what we mean.

Flower	Length (cm)	Width (cm)	Known species	Row ID
A	4.6	1.5	versicolor	54
B	4.5	1.7	virginica	106
C	4.7	1.4	versicolor	50
D	5.0	1.5	virginica	119
E	5.1	1.5	virginica	133

Output: the species name

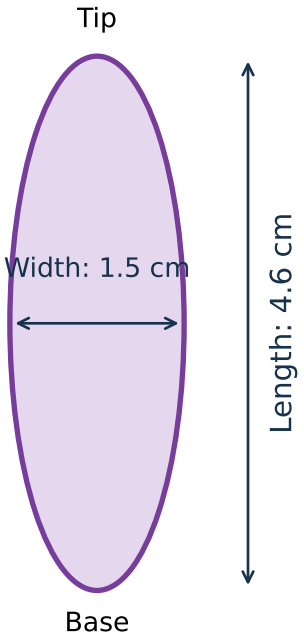
The known species is the target or class label, not a third measurement. Row IDs are zero-based positions in sklearn's dataset, not features. We select these rows to teach the arithmetic, not to measure model quality.

Our query flower has length 4.5 cm and width 1.5 cm (row 51, excluded from the five references). Hide its known species until after prediction.

What exactly do we measure?

A petal is a colored part of a flower. This simplified sketch shows one petal, not a whole flower.

For flower A, the dataset records a petal length of 4.6 cm and a petal width of 1.5 cm. Imagine placing a ruler along each direction below. These are physical sizes, not ratings or points.



From a ruler to a point on a graph

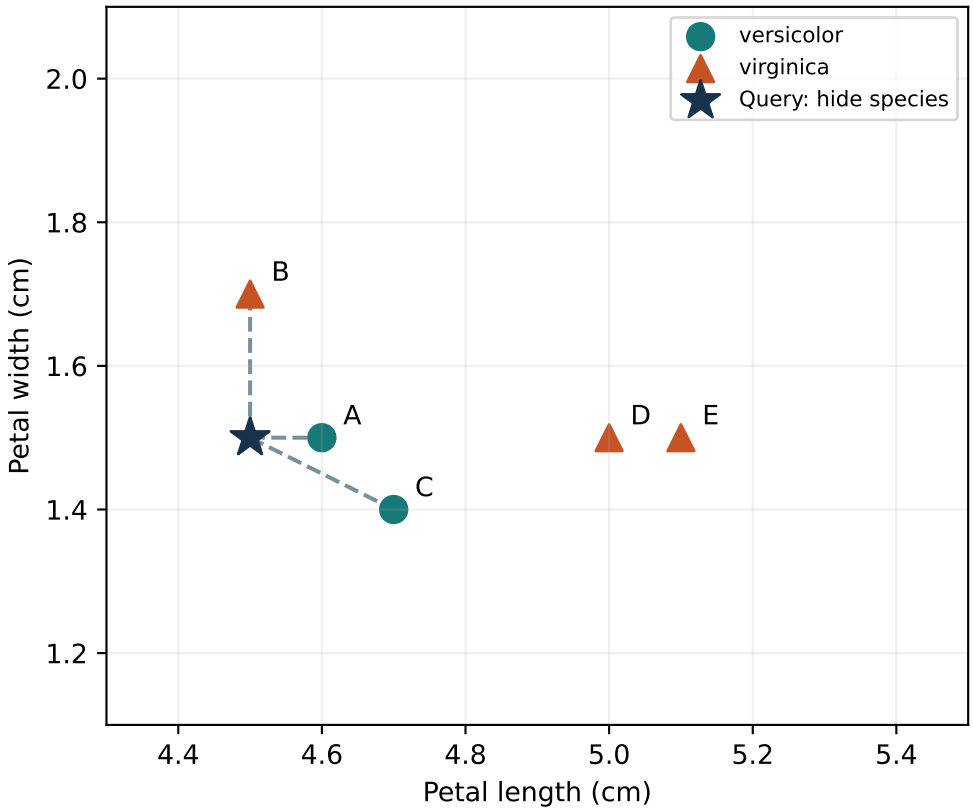
Flower A becomes the coordinate (4.6, 1.5): length first, width second. Our query flower becomes (4.5, 1.5). KNN compares these measurements to find similar reference flowers.

Illustrative measurement sketch, not a botanical drawing of an actual specimen.

A prediction by nearby examples

KNN means k-nearest neighbors. k is how many neighbors vote, not how many classes exist.

Each colored point is a reference flower from page 1. Position shows petal length and width; color and shape show known species. The star is the query at (4.5, 1.5). The plot is zoomed in; both axes use equal centimeter spacing.



With k = 3, use A, B, and C

A and C are versicolor; B is virginica. Two versicolor votes beat one virginica vote, so predict versicolor. Do not average species codes as though they were measurements. Next, we calculate the distances.

Watch two distance calculations

Euclidean distance means straight-line distance. We show every step before you try one.

The query is (4.5, 1.5). Here x is a reference flower's petal length and y is its petal width, both in cm. d is distance in this two-measurement plane. Species is NOT used in this calculation.

$$d = \sqrt{(x - 4.5)^2 + (y - 1.5)^2}$$

A square multiplies a number by itself: 0.1 squared is 0.01. A square root reverses squaring. Keep the calculator's full precision when ordering distances; round only for display.

Worked example 1: A is at (4.6, 1.5)

Subtract: 4.6 minus 4.5 is 0.1; 1.5 minus 1.5 is 0.

$$d_A = \sqrt{(4.6 - 4.5)^2 + (1.5 - 1.5)^2}$$

$$d_A = \sqrt{0.1^2 + 0^2} = \sqrt{0.01} = 0.1$$

A is 0.1 cm away in the length-width plane.

Worked example 2: C is at (4.7, 1.4)

Subtract: 4.7 minus 4.5 is 0.2; 1.4 minus 1.5 is -0.1.

$$d_C = \sqrt{0.2^2 + (-0.1)^2} = \sqrt{0.04 + 0.01}$$

$$d_C = \sqrt{0.05} \approx 0.224$$

A negative difference becomes positive when squared. A is closer than C.

Watch the neighbors vote

First calculate distances. Then sort. Only then use the known species to vote.

Here are all five distances to the query flower at (4.5, 1.5), sorted from nearest to farthest. Distances use raw centimeters for this small arithmetic demonstration.

Flower	Known species	Distance (cm)	Votes at k = 3?
A	versicolor	0.100	YES
B	virginica	0.200	YES
C	versicolor	0.224	YES
D	virginica	0.500	NO
E	virginica	0.600	NO

Worked vote: choose k = 3

Take the first three rows: A, B, C. Their species are versicolor, virginica, versicolor. Two versicolor votes beat one virginica vote. Predict versicolor. D and E do not vote.

How much of the vote was for versicolor?

$$\text{Versicolor vote fraction} = \frac{2}{3} \approx 0.667 = 66.7\%$$

Two of three chosen neighbors are versicolor. This is a neighbor fraction, not guaranteed real-world confidence. We deliberately selected five references; this is not an accuracy study of all 150 flowers.

Why distances lead to votes

Distance chooses who participates. The selected neighbors' labels choose the prediction.

1. Calculate, sort, and select

Calculate distances to the reference flowers and sort smallest to largest. With k = 3, keep ONLY the first three: A, B, C. D and E are farther away and do not participate in this prediction.

2. Count the known species

We want a species, not a distance. 'Voting' means counting labels: A and C are versicolor; B is virginica. Two versicolor votes beat one virginica vote. Return versicolor, then stop.

Why not use only the single closest flower?

That is exactly k = 1. A small neighborhood can be less sensitive to one noisy or unusual reference. More neighbors do not guarantee better predictions; choose k using validation data.

What is the 66.7% for?

It is optional extra information: 2 of 3 chosen neighbors are versicolor. Counting 2 versus 1 already gives the prediction. Report: 'Predicted versicolor; 2 of its 3 nearest references are versicolor.'

The percentage does NOT feed back into training, adjust measurements, or start another iteration in our workflow. It is not necessarily a calibrated real-world probability. With equal votes, A and C each count once despite their different distances. Distance-weighted KNN uses a different rule.

The algorithm before Python

Pseudocode describes the steps in plain language. This is classification with equal-weight votes.

Fit: prepare the reference information

INPUT: training measurements, known labels, and a chosen k.
1. Learn any preprocessing statistics from training rows only.
2. Apply that preprocessing to the training measurements.
3. Store the processed measurements and their labels.
4. Fitting is complete; a library may also build a search index.

Predict: process one new row

1. Apply the saved preprocessing to the new measurements.
2. Calculate distances to the stored training examples.
3. Sort from smallest distance to largest.
4. Select the first k examples.
5. Count how many selected examples belong to each class.
6. Return the class with the most votes. Prediction is complete.
7. Optionally return each class's count divided by k.

This describes a straightforward distance search. Libraries can use optimized searches without sorting every reference explicitly. The prediction still depends on the selected k nearest neighbors.

The hand example uses raw centimeters; the main notebook saves and applies a scaler. The unknown row's true species is never an input to prediction. A complete implementation also needs rules for tied votes and equal-distance neighbors; our worked example has neither tie.

When does KNN stop training?

Basic KNN has no repeated coefficient updates and no loss-minimization loop like logistic regression.

Fitting is not the same as selecting k

Logistic regression repeatedly adjusts coefficients to reduce a loss until its solver stops. Basic KNN retains examples and labels; fitting ends after preparing/storing this reference data and any search structure. It is often called lazy learning: much of the work happens at prediction time.

Model selection: a separate loop in our notebook

FOR each k in the specified candidate list:
Fit preprocessing and KNN using TRAINING rows.
Predict VALIDATION rows and measure validation accuracy.
Choose the k with the highest validation accuracy.
If scores tie, our notebook chooses the smaller k.
STOP after all specified candidate values have been evaluated.

Maximizing validation accuracy is equivalent to minimizing its fraction of incorrect predictions. This selects a hyperparameter, not coefficients. It does not keep changing the reference measurements until every error disappears.

Freeze settings, refit, then test

After choosing k, fit a fresh model on training + validation rows. Evaluate it once on untouched TEST rows. Do not select k using test results. Test performance estimates new-data performance only when future data is sufficiently similar.

Your turn: calculate, then vote

Follow the worked examples on pages 4-5. Show your reasoning before checking with your instructor.

1. Calculate two distances to (4.5, 1.5)

B is at (4.5, 1.7). D is at (5.0, 1.5). For each one: subtract coordinates, square the differences, add them, and take the square root. Show all steps.

B: _____

D: _____

2. Change the number of voters

Use the sorted table on page 5. Predict for k = 1 and k = 5. Count each species's votes. Does the prediction change from the worked k = 3 example?

My votes and predictions: _____

Explain your reasoning

Which reference flowers vote for each choice of k?

_____

Why can the prediction change when the flower has not changed?

_____

How is this a trained model?

Basic KNN retains training examples; it does not learn logistic coefficients or decision-tree rules.

1. Split before preprocessing

Training rows build the model. Validation rows help select k. Reserve test rows for one final evaluation.

2. Fit the scaler on training rows

The scaler puts features on comparable scales using training means and standard deviations. The next page shows the arithmetic. Never fit it on test rows.

3. Fit KNN

Retain the scaled training features and known labels, with settings such as k, distance metric, and vote weighting.

4. Predict a new row

Use the saved scaler, find k training neighbors, and vote. Do not refit the scaler on the new row or add its unknown label to training.

5. Interpret the vote fraction carefully

With equal weights, 2 versicolor votes out of 3 gives a vote fraction of two thirds. It is not proof of the species or a guaranteed calibrated probability.

Watch a scaling calculation

A separate invented example: these numbers are NOT the course dataset's fitted statistics.

Features can have different units and spreads. Standardization measures how far a value is above or below the training mean, in training standard-deviation units.

$$z = \frac{\text{value} - \text{training mean}}{\text{training standard deviation}}$$

Given: summaries already learned from training rows

Feature 1: training mean = 10, training standard deviation = 2.
Feature 2: training mean = 20, training standard deviation = 5.
New row: feature 1 = 14; feature 2 = 25.

Substitute, subtract, then divide

$$z_1 = \frac{14 - 10}{2} = \frac{4}{2} = 2$$

$$z_2 = \frac{25 - 20}{5} = \frac{5}{5} = 1$$

The raw row (14, 25) becomes (2, 1). Feature 1 is two standard deviations above its training mean; feature 2 is one above. KNN calculates distances between scaled rows using these same saved training summaries.

Case study: 85 million flowers

HYPOTHETICAL: all large-scale counts, records, scores, and results in this case are invented.

Imagine a flower-monitoring company has 85,000,000 labeled records. For this story, experts have confirmed three known species: setosa, versicolor, virginica. We predict the species of a new flower from petal length and width in centimeters.

Record	Length (cm)	Width (cm)	Expert species
F001	1.4	0.2	setosa
F002	4.7	1.4	versicolor
F003	5.5	2.0	virginica
F004	5.1	1.8	virginica
F005	4.9	1.5	versicolor

These five fake records are a window into the larger data, not the whole training set. They are separate from the real Iris rows used earlier in the guide.

What KNN IS for

Supervised classification: assign a new flower to a class present in labeled references. Ordinary KNN still chooses a known class for an unfamiliar species; it does not automatically discover or reject new species, or explain biological causes.

What if nobody knows how many species exist?

KNN classification cannot discover the number of species. HDBSCAN clustering can suggest groups without fixing their count in advance, but groups are NOT proof of species. Experts must examine biology and confirm labels. Explore a representative sample first; 85 million rows still requires a scaling plan.

Split before choosing anything

Training fits the model. Validation selects settings. Test is the final independent check.

Assume each record is a different plant and the intended task is predicting similar new plants. Use a species-stratified 60/20/20 split. If plants have repeat records, keep each plant in one split; for future dates or new farms, use a matching time/group holdout instead.

Pool	Rows	Purpose
Training	51,000,000	Fit scaler + references
Validation	17,000,000	Choose k; diagnose errors
Test	17,000,000	Final evaluation only

Record	Assigned pool	How its label is used
F001	Training	Fit reference
F002	Training	Fit reference
F003	Training	Fit reference
F004	Validation	Check prediction only
F005	Test	Check prediction only

Use ALL rows in this hypothetical, in their proper roles

All 51 million training rows are references during tuning. All 17 million validation rows help choose k. All 17 million test rows stay locked until final evaluation. We display only five records; they do not replace the full data. Process queries in batches rather than loading every query-to-reference distance at once.

What did fitting actually learn?

Prepare all 51 million training references; the five shown rows are only a preview.

Check units, impossible values, duplicate plants, and label quality first. If imputation is needed, fit it using training rows only. Here all shown measurements are present. Suppose the training scaler learns these invented summaries:

$$z_L = \frac{L - 4.0}{1.5} \qquad z_W = \frac{W - 1.0}{0.5}$$

L and W are raw length and width in cm. Means are 4.0 and 1.0; standard deviations are 1.5 and 0.5. These summaries come from all 51 million training rows, NOT from the three displayed training rows.

Record	Scaled length	Scaled width	Pool
F001	-1.733	-1.600	Training
F002	0.467	0.800	Training
F003	1.000	2.000	Training
F004	0.733	1.600	Validation
F005	0.600	1.000	Test

F005's values above are an illustration only; actual test transformation waits for the final frozen pipeline.

What is learned, stored, and NOT learned?

The scaler learns training means and spreads. Basic KNN stores scaled training measurements and labels, plus k and distance/voting rules; it may build a search index. It does not learn coefficients, a species-discovery rule, or a loss curve. F004 is transformed for validation, not stored as a neighbor during tuning. F005 stays locked.

Choose k using validation

HYPOTHETICAL scores: this is a worked selection example, not a result of running 85 million rows.

Before comparing results, specify uniform votes, Euclidean distance, scaling, candidate k values, and validation accuracy as the selection metric. Smaller k breaks score ties. Each candidate uses the SAME 51 million training and 17 million validation rows.

k	Train accuracy	Validation correct	Val accuracy
1	100%	13,940,000	82%
3	98%	14,790,000	87%
7	96%	15,300,000	90%
15	94%	15,130,000	89%
51	86%	14,280,000	84%

Decision: select k = 7

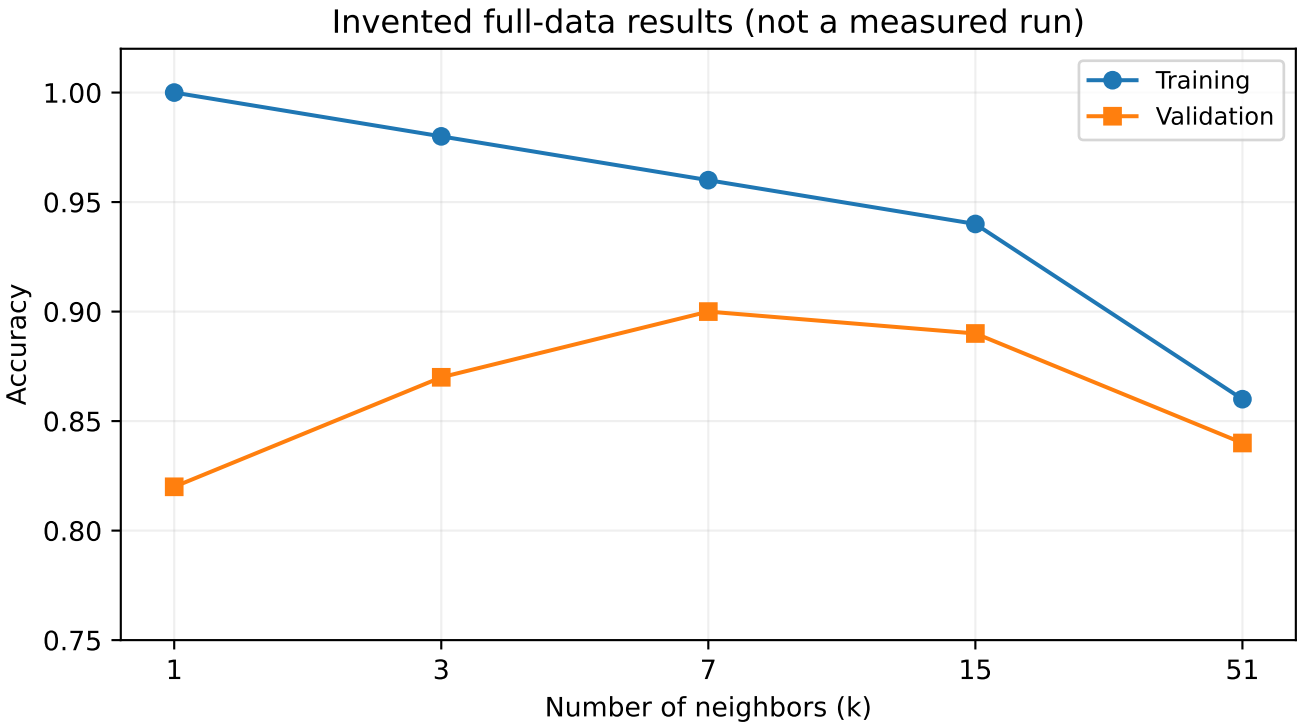
15,300,000 of 17,000,000 validation flowers are correct: 90%. This is the highest validation accuracy among these candidates. Equivalently, 1,700,000 are wrong: a 10% validation error rate. Do NOT choose k = 1 just because its training score is 100%.

Validation labels act as an answer sheet AFTER prediction, never as voting neighbors. F004 participates in validation scoring; F005's test label remains untouched. Choosing k is hyperparameter tuning, not coefficient training.

Search once for 51 neighbors per query; evaluate the first 1, 3, 7, 15, or 51. This reuses distances. Accuracy is our predeclared objective; rare-species projects may preselect recall or macro-F1 instead.

Overfitting and knowing when to stop

KNN can overfit even though it does not repeatedly adjust coefficients.



Read the gap, not just the training score

k = 1: 100% training vs 82% validation is an 18-percentage-point gap, a warning of noise sensitivity. A training row can be its own nearest neighbor. k = 7: 96% vs 90% is a smaller 6-point gap and better validation performance.

k = 51 has a small 2-point gap but lower scores (86% / 84%): that may be underfitting. A small gap alone is not success. Gaps are clues, not proof; check group leakage, repeated splits, learning curves, and class-specific errors using development data.

Fitting finishes when preprocessing and reference storage/index construction finish. Tuning finishes after all planned k values are scored and k = 7 is frozen. There is no 'keep reducing KNN loss until convergence' step. Further experiments use validation, not test results.

Follow F004 through a prediction

During validation, F004 is a query only. Its true species is hidden until after the vote.

$$(5.1,\ 1.8) \longrightarrow \left(\frac{5.1-4.0}{1.5}, \frac{1.8-1.0}{0.5}\right) \approx (0.733,\ 1.600)$$

Search among ALL 51 million scaled training references. Keep the SEVEN closest; only those seven vote. An index may find them without scanning every reference. Here is an invented report of the neighbors selected for this query:

Rank	Reference ID	Scaled distance	Known species
1	N11	0.03	virginica
2	N12	0.07	versicolor
3	N13	0.13	virginica
4	N14	0.20	virginica
5	N15	0.30	versicolor
6	N16	0.33	virginica
7	N17	0.45	versicolor

Return the winning species, then stop

Virginica wins 4 votes to 3; no setosa reference is selected. Predict virginica. Its vote fraction is 4/7, about 57.1%: an optional description of neighbor agreement, not a training update or reliable calibrated probability. Odd k does not prevent every multiclass tie; this vote has no tie.

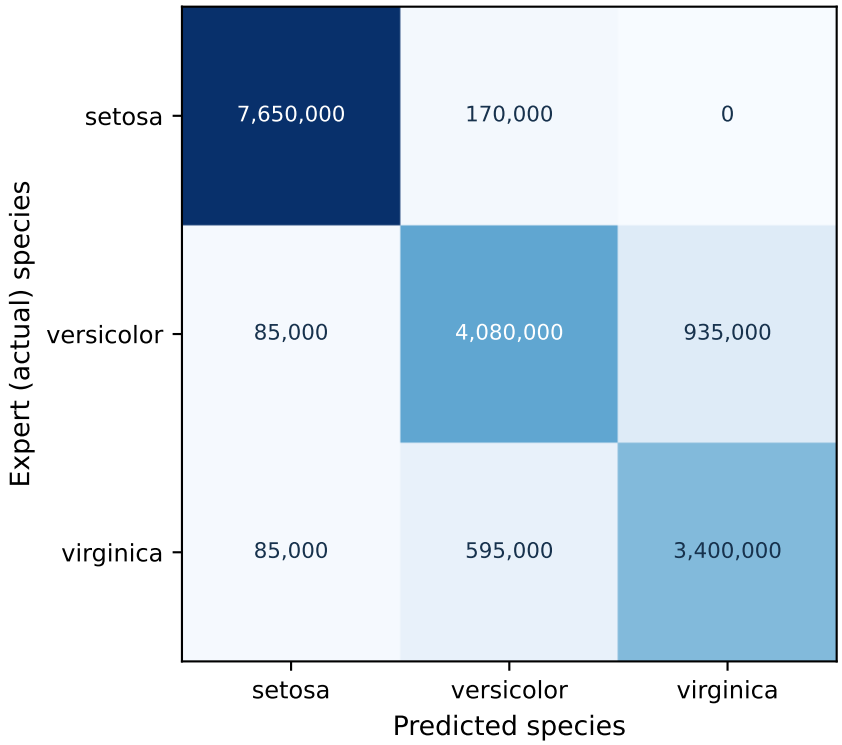
Reveal F004's label: virginica. Count one correct prediction. Score ALL 17 million validation queries, not just this convenient example.

Freeze, refit, then open the test set

The final evaluation model has 68 million references. Test labels never become neighbors.

Freeze k = 7 and all other settings. Fit a NEW scaler and KNN on the combined 51 million training + 17 million validation rows. F004 can now be a reference. The scaler's summaries may change; do not reuse the previous page's invented values blindly. F005 remains test-only.

Apply the final saved pipeline to all 17 million locked test flowers, then compare predictions with expert labels once. F005 is just one of these rows. Here is an invented confusion matrix for the final model:



Read one row: actual virginica

Of 4,080,000 actual virginica flowers, 3,400,000 are predicted virginica, 595,000 versicolor, and 85,000 setosa. So 680,000 are missed. Off-diagonal cells are errors; diagonal cells are correct.

What do these scores mean?

Hypothetical test results: the denominators tell you which question each metric answers.

Accuracy: 89.0% overall

15,130,000 of 17,000,000 flowers are correct; 1,870,000 are wrong. This beats a 46% always-setosa baseline on this test set, but does not tell us which species is being missed.

Virginica precision: 3,400,000 / 4,335,000 = 78.4%

Of flowers CALLED virginica, about 78 in 100 really are virginica. There are 935,000 false positives. This matters when assigning the wrong species is costly. It does NOT mean 78% of all virginica flowers were found.

Virginica recall: 3,400,000 / 4,080,000 = 83.3%

Of actual virginica flowers, about 83 in 100 are detected; 680,000 are missed (false negatives). This matters when failing to find a species is costly. Precision and recall use different denominators.

Virginica F1: 6,800,000 / 8,415,000 = 80.8%

F1 balances precision and recall using their harmonic mean; it is not overall accuracy. Report every species. Macro-F1 averages the species F1 scores equally so common species do not dominate.

Analyst statement: 'Overall accuracy is 89%, but we miss 680,000 virginica flowers and incorrectly call 935,000 other flowers virginica. Whether this is acceptable depends on the costs of those errors, not a universal passing score.'

Do not change k because you dislike the test score. Improve using development data, then obtain a fresh final evaluation if test findings influenced your decisions.

Do we really scan 85 million rows?

No fixed universal row limit. Practical limits are memory, search time, hardware, and implementation.

Eligible references are not the same as rows examined

During tuning, 51 million training rows are eligible neighbors. After refitting, 68 million are eligible. Brute force checks EVERY eligible reference per query. An exact tree-based index can prune candidates and still return exact neighbors, but may approach a full scan in difficult data.

Why brute force gets expensive

51 million references x 17 million validation queries = 867 trillion distance comparisons for one exhaustive pass. Reuse the nearest-51 results for all candidate k values. Batch queries and stream data; never allocate the full query-by-reference distance matrix.

85 million rows x 2 float64 features x 8 bytes = 1.36 GB (decimal) for raw features alone. Labels, IDs, scaler copies, indexes, and search buffers add memory. Fitting in memory does NOT prove prediction will meet a response-time target.

A realistic engineering decision, not a magic setting

Benchmark exact indexes (especially plausible for only two features) against brute force. Approximate-neighbor indexes trade exactness for speed and can change votes: measure both neighbor-retrieval recall and classification metrics on validation. A smaller representative reference sample is another option, but changes the model and must be validated.

After evaluation, you may refit on all 85 million rows for deployment. The old score evaluated 68 million references, not this new model. Those rows are no longer an independent test. Assess deployment on fresh labeled data and monitor drift and latency.

Yes, use batches for large data

Batching controls memory and scheduling. It is NOT a KNN gradient-descent training loop.

Load and preprocess in chunks

Read the training pool in chunks, for example 1 million rows at a time. Training-only running statistics can produce one GLOBAL scaler (StandardScaler supports partial_fit). Then transform chunks with that saved scaler. Do not fit a different scaler for each chunk.

Build or store a searchable representation of ALL 51 million scaled training references. Ordinary sklearn KNeighborsClassifier has no partial_fit: repeatedly calling fit on chunks replaces the references; it does not accumulate them. Large-scale storage/index construction requires suitable tooling.

Validation batch	Query rows	Eligible reference set
1	1 - 10,000	All 51 million training rows
2	10,001 - 20,000	All 51 million training rows
...	...	Same reference set
1,700	16,990,001 - 17,000,000	Same reference set

For each query batch, find up to 51 neighbors per flower and score each candidate k. Add correct-prediction counts to running totals. Only after ALL 1,700 batches, divide each total by 17 million and select k. Do not choose a different k per batch or update weights.

Exact brute-force search can also scan reference blocks and merge the nearest candidates across ALL blocks. Searching only the current block gives the wrong neighborhood. An index can reduce scanning; batching alone does not reduce the number of exhaustive comparisons.

Bound internal search buffers too; query batch size alone is not a memory guarantee. After refitting, batch the test queries against all 68 million references without tuning on them.

Now connect the idea to code

The companion notebook runs the full experiment. This excerpt assumes training and validation arrays exist.

The Iris hand example ends here. For the main code comparison, return to the course breast-cancer data: mean radius and mean texture, with 0 = benign and 1 = malignant. This is not for patient care.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

knn = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5,
                         weights="uniform"),
)

knn.fit(X_train, y_train)
predictions = knn.predict(X_validation)
vote_fractions = knn.predict_proba(X_validation)
```

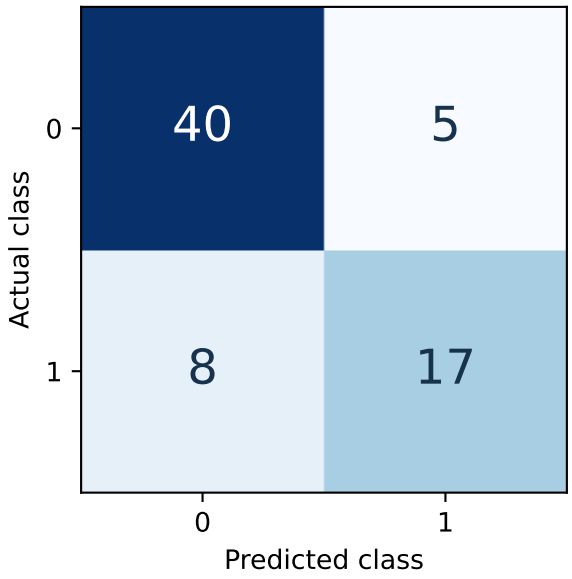
Read the code in plain English

make_pipeline connects scaling to KNN. fit learns the scaler and stores the training examples. predict applies both steps to new raw measurements. predict_proba returns fractions for the classes in classes_ order.

How well did it perform?

Count mistakes on unseen rows. The example below is illustrative, not a measured notebook result.

Here we switch to the medical-data labels: 0 = benign, 1 = malignant.



Of 70 rows, 40 + 17 are correct. For class 1 there are 17 correct detections, 8 missed detections, and 5 false alarms.

$$\text{Accuracy} = \frac{\text{correct}}{\text{all}} = \frac{40+17}{70} \approx 81.4\%$$

$$\text{Precision} = \frac{\text{correct class 1}}{\text{predicted class 1}} = \frac{17}{17+5} \approx 77.3\%$$

$$\text{Recall} = \frac{\text{detected class 1}}{\text{actual class 1}} = \frac{17}{17+8} = 68.0\%$$

$$\text{F1} = \frac{2 \times 17}{2 \times 17 + 5 + 8} \approx 72.3\%$$

F1 balances precision and recall using their harmonic mean.

Interpretation: "The model missed 8 of 25 actual malignant rows. Accuracy alone hides these misses."

What to watch for

KNN is easy to picture, but a meaningful idea of closeness is essential.

Choosing k

Small k is sensitive to noise; k = 1 can memorize training rows. Very large k can blur patterns and favor the majority class. Select k using validation, not test scores.

Missing values and categories

sklearn KNN does not accept NaN directly. Fit imputation inside a pipeline using training data only. Category codes such as 1, 2, 3 do not automatically represent meaningful distances.

Ties, weighting, and boundaries

Odd k helps avoid binary vote ties with uniform weights, but does not prevent equal-distance ties at the neighbor cutoff. Distance-weighted voting is a different rule from the equal votes taught here.

Many features and large datasets

Irrelevant features can distort closeness. High-dimensional distances can be less informative. Retaining and searching training examples requires memory and prediction time.

What the notebook's plots tell you

Neighbor plots explain one vote; decision boundaries show predicted regions. Compare training and validation curves for overfitting, then confusion matrices for error types.

Sources: scikit-learn estimator and dataset documentation; links in the notebook.